السلام عليكم ورحمة الله وبركاته.. وبعد،

بين يديك ملخص شامل ومنظم لمادة Chapter5 _ CPCS-241، تم إعداده بعناية ليكون عوناً لك في فهم المادة .وقبل البدء، أضع بين يديك ميثاقاً للأمانة العلمية والتعاون:

⚠️ ميثاق الأمانة والمسؤولية:

هذا العمل هو جهد طالب، أضعه بين يديك بنية النفع والفائدة؛ لذا:

- لا أسامح ولا أحلل استخدامه في الغش أو أي عمل ينافي الأخلاق الإسلامية.
- يُمنع منعاً باتاً تداوله كمصدر تجاري للباعة أو استخدامه في أدوات الذكاء الاصطناعي لغرض التحايل او الغش.
- الأمانة العلمية تقتضي منك استخدامه كمرجع للفهم والمراجعة، لا كبديل عن الضمير والاجتهاد الشخصي.

🌐 خدمات إضافية عبر الموقع:

يمكنك تعزيز مذاكرتك من خلال الوصول إلى أسئلة تفاعلية تجريبية للمنهج عبر الرابط: 🔗 kau-one.web.app/quiz

---

📣 التواصل معي:

الكمال لله وحده، لذا يسعدني استقبال ملاحظاتكم، تعديلاتكم، أو إضافاتكم عبر:

- الرقم المباشر: 0530717788
- الموقع الشامل: kau-one.web.app

توجيه إيماني:

قبل أن تبدأ، استعذ بالله من الشيطان الرجيم، وجدد نيتك بأن يكون هذا الوقت في سبيل طلب العلم النافع الذي يرفع قدرك في الدنيا والآخرة. قال رسول الله ﷺ: "مَنْ سَلَكَ طَرِيقًا يَلْتَمِسُ فِيهِ عِلْمًا سَهَّلَ اللَّهُ لَهُ بِهِ طَرِيقًا إِلَى الْجَنَّةِ" (رواه مسلم).

إعداد الطالب: إلياس حسان بابور الدفعة: 24 — تم اخذ الماده و عمل الملخص على السلايدات سنه م2026

Relational Model Concepts

- **Theldea:** This model is based on the mathematical concept of a **Relation**.
- **History:** It was created by **Dr. E.F. Codd** at IBM in 1970. His paper changed the world and won him the Turing Award (the "Nobel Prize" of Computing).
- **Formal vs. Practical:** There is a **Formal** model (math) and a **Practical** model (SQL). They have slight differences, which we will see.

Informal Definitions

To understand the math, first look at it informally:

- A **Relation** looks like a **Table** of values that typically contains a set of rows .
- A **Row** represents a real-world entity or relationship (like one student).
    - In the formal model, rows are called tuples
- A **Column Header** gives meaning to the data (in the formal model this is the **Attribute**).

The Key of a Relation

- Each row must be **unique**.
- **The Key:** The data item that identifies the row (e.g., SSN).
- **Artificial / Surrogate Key:** If there is no natural key, we invent a sequential number (like `Row-ID`) to be the key.

Formal Definitions: Schema, Tuple, & Domain

- **Schema (R):** The description. Written as: R(A_1, A_2, ..., A_n).
    - *Example:* CUSTOMER(Cust-id, Cust-name).
- **Tuple:** A formal word for a **Row**.
    - Written in angled brackets: <632895, "John Smith”}> this is 2-tuple
    - If it has 4 values, it is a **4-tuple**.
- **Domain (D):** The set of valid values .
    - *Example:* A domain of "USA_phone_numbers" is the set of all valid 10-digit numbers.

Formal Definitions: State & Cartesian Product

- **Cartesian Product :** This is a set of **all possible** combinations of values.
- **Relation State (r(R)):** This is the **actual population** of the table at a specific moment. It is always a *subset* of the Cartesian Product.

Example (Slide 5-14):

If Domain 1 = {0, 1} and Domain 2 = {a, b, c}

The **Cartesian Product** is all 6 pairs: {<0,a>, <0,b>, <0,c>, <1,a>, <1,b>, <1,c>}.

The **Relation State** is just the ones we pick to be in our table, like: {<0,a>, <1,c>}.

Definition Summary Table

Memorize this for the exam

Informal Term	**Formal Term**
Table	**Relation**
Column Header	**Attribute**
Possible Column Values	**Domain**
Row	**Tuple**
Table Definition	**Schema of a Relation**
Populated Table	**State of the Relation**

اللهم اجعلنا من الذاكرين الشاكرين.

Characteristics of Relations (slide 17)

- **Tuple Ordering:** The **order of rows does not matter**. If you swap the first row with the last row, the "State" of the relation is exactly the same.
- **Attribute Ordering:** In the formal model, we usually consider the **order of columns to be fixed** in the Schema R(A_1, A_2, ..., A_n).
    - *Self-describing alternative:* Some models include the attribute name with the value (e.g., {Name: 'Elyas', Age: 20}), which means order wouldn't matter there either.
- **Values are Atomic:**Every value in a tuple must be **indivisible**. You cannot have a list or an array as a single value (e.g., no Colors: [Red, Blue] in one cell).
- **NULL Values:** If a value is unknown, not available, or doesn't apply, we use a special **NULL**.
- **Notation:** t[A_i] means: "Give me the value of attribute A_i for row t."

The Three Types of Constraints

1. **Inherent (Implicit):** Built into the model (e.g., "relational model does not allow a list as a value for any attribute").
2. **Schema-based (Explicit):** You define these in the code (e.g., "Salary cannot be negative" or "SSN must be unique").
3. **Application-based (Semantic):** Handled by the software, not the DB (e.g., "A student cannot register for more than 5 courses").

Relational Integrity Constraints

Constraints are conditions that must hold on all valid relation states.
There are three main types of **(explicit schema-based)** constraints that can be expressed in the relational model:

- Key constraints
- Entity integrity constraints
- Referential integrity constraints

Another schema-based constraint is the domain constraint Every value in a tuple must be from the domain of its attribute (or it could be null, if allowed for that attribute)

اللهم ثبتنا على دينك حتى نلقاك.

Key Constraints: Superkey vs. Key

- **The Superkey**
  - A **Superkey** is any group of attributes that can distinguish one row from another.
  - **Fact:** Because your SSN is unique, `{SSN}` is a superkey.
  - **The "Super" Part:** If you add more info, like `{SSN, Name}`, it is **still** unique. You can identify the person with just the SSN, but adding the Name doesn't stop it from being unique.
  - **Logic:** `{SSN, Name, Address, EyeColor}` is a **huge** superkey. It's "overkill," but it works.
  -
- **The Key**
  - A **Key** is a **Minimal Superkey**. This means you have *just enough* information to be unique, and not a single drop more.
  - **Example:** `{SSN}` is a Key. If you remove `SSN`, you have nothing left.
- **Example from Slide 5-24:** `{SerialNo, Make}` is a Superkey. But if you remove `Make`, you are left with `{SerialNo}`, which is still unique! Therefore, `{SerialNo, Make}` was **not** a Key—it was just a Superkey.

Rule to Remember: A Key is a Superkey, but a Superkey is not always a Key. (Because a Superkey can be "more than one" with extra, useless data).

Candidate vs. Primary Key

Sometimes a table has multiple keys (multiple ways to be unique).

- **Candidate Keys:** All the possible "Minimal" keys. In the CAR example, both `{State, Reg#}` and `{SerialNo}` are Candidate Keys.
- **Primary Key:** The one you choose to be the "Boss."
  - **The Choice:** Usually, you pick the smallest one (like `SerialNo`).
  - **The Drawing:** You **Underline** it.

اللهم ارزقنا راحة البال وسكينة القلب.

Relational Database Schema (S)

- **What it is:** A collection of multiple **Relation Schemas** (R_1, R_2, ... R_n) that all belong together.
- **The Law (IC):** It also includes the **Integrity Constraints**. These are the rules that keep the data clean and connected.
- **The Example:** The COMPANY schema isn't just one table; it's a set of 6 tables: EMPLOYEE, DEPARTMENT, PROJECT, etc., all working together.

Relational Database State (DB of S)

- **What it is:** The actual data (the "population") inside those tables at a specific moment.
- **The Condition:** For a state to be **Valid**, the data in *every* table must follow the laws (IC) defined in the Schema.
- **Invalid State:** If you have a row that breaks a rule (like a negative salary or a missing ID), the whole Database State becomes **Invalid**.
- **Snapshot:** Just like taking a photo. Every time you change one piece of data, you create a **new state**.

How the State Changes

The state evolves through three basic operations:

1. **INSERT:** Adding a new tuple (row).
2. **DELETE:** Removing a tuple.
3. **MODIFY:** Changing a value in an existing tuple.

Integrity

(importnat)

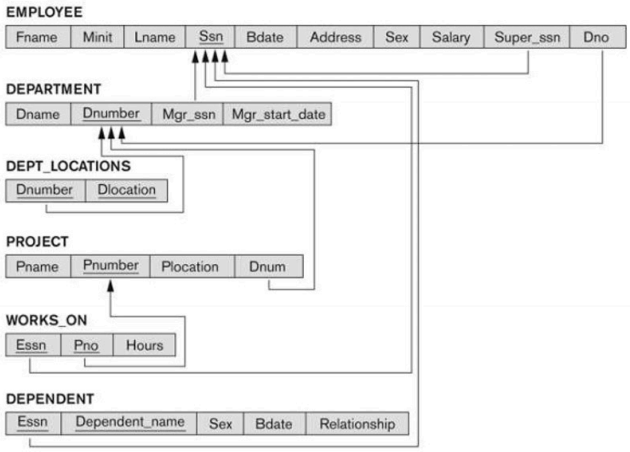
- **Rule 1: Entity Integrity(single relation):** The **Primary Key** cannot be **NULL**.
  - *Why?* Because if the "ID" is empty, we can't identify the row.
- **Rule 2: Referential Integrity (Foreign Keys):** This involves **two** tables.
  - A **Foreign Key (FK)** in Table A points to a **Primary Key (PK)** in Table B.
  - **The Rule:** The value in the FK must either:
    1. Match an existing PK value in the other table.
    2. Be **NULL** (unless it's part of its own PK).

اللهم صل وسلم على نبينا محمد.

Drawing the Relational Schema

- **The Table:** Draw a long, thin rectangle (a row) and divide it into boxes for each attribute.
- **The Name:** Write the Table Name (e.g., `EMPLOYEE`) **above** the boxes.
- **The Primary Key: <>Underline<>** the attribute(s) inside the boxes.
- **The Foreign Key Arrow:** Draw an **arc (arrow)** starting from the Foreign Key and pointing to the Primary Key of the table it references.

exable :



Semantic Integrity Constraints

These are "Special Rules" that the standard model (Keys and Nulls) can't handle alone.

- **The Logic:** Rules like *"An employee cannot work more than 56 hours"* aren't about uniqueness; they are about business logic.
- **The Tools:** We use **TRIGGERS** or **ASSERTIONS** in SQL to enforce these.

Update Operations

1. Restrict (Reject)

- Cancels the operation that causes the violation**,**the database does not allow the change to proceed

2. Allow with Warning

- Performs the operation,but informs the user that a constraint violation occurred

3. Automatic Correction (Integrity Actions)

The database automatically fixes the violation by modifying related data(cascade,set null)

4. User-Defined Action

- Executes a custom error-handling routine defined by the user or application

Possible violations for each operation

1. INSERT

Adding a new row is risky because it can violate **every single type** of constraint:

- **Domain Constraint:** You try to put "Apples" in a "Price" column that only accepts numbers.
- **Key Constraint:** You try to add a student with ID 123, but ID 123 already exists.
- **Entity Integrity:** You try to add a student but leave the Primary Key (ID) empty (**NULL**).
- **Referential Integrity:** You try to add an employee to "Department 99," but "Department 99" doesn't exist in the Department table.

2. DELETE

Deleting is much simpler. It **only** risks violating **Referential Integrity**.

- **The Scenario:** If the primary key value of the tuple being deleted is referenced from other tuples in the database
- **The Fix:**
  - **RESTRICT:** Block the delete.
  - **CASCADE:** propagate the new primary key value into the foreign keys of the referencing tuples
  - **SET NULL:** set the foreign keys of the referencing tuples to NULL

One of the above options must be specified during database design for each foreign key constraint

3. UPDATE

An update's risk depends on **what** you are changing:

- **Updating a Primary Key:** This is treated like a DELETE followed by an INSERT. It is very risky and can break almost anything.
- **Updating a Foreign Key:** Can break **Referential Integrity**
- **Updating a Regular Attribute (like Name):** Usually only risks breaking a **Domain Constraint** (if you put the wrong data type).

i skipped the summary and In-Class Exercise to see them slides : 44-45

اللهم اغفر لنا ولوالدينا ولجميع المسلمين.

ملاحظة ختامية

الحمد لله الذي بنعمته تتم الصالحات، والصلاة والسلام على معلم البشرية الخير، نبينا محمد وعلى آله وصحبه أجمعين.

تم بفضل الله وتوفيقه الانتهاء من نهائيه الملخص

رسالة ختامية: كما ذكرتُ في المقدمة، فإن هذا الجهد أمانة بين يديك؛ فلا تجعل استخدامه يخرج عن إطار طلب العلم ، وابتعد عن كل ما يخدش ضمير الطالب المسلم من غش أو استغلال غير أخلاقي.

تذكير أخير: ما كان في هذا الملخص من صواب فمن الله وحده، وما كان فيه من خطأ أو نسيان فمني ومن الشيطان. وكما هو موضح في بيانات التواصل، بابي مفتوح لكل تصحيح أو إضافة تثري هذا العمل وتخدم الدفعات القادمة.

دعاء: اللهم انفعنا بما علمتنا، وعلمنا ما ينفعنا، وزدنا علماً. اللهم اجعل ما قرأناه حجة لنا لا علينا، ووفقنا جميعاً في اختباراتنا وفي حياتنا العملية لخدمة دينه ووطننا.

إعداد الطالب: إلياس حسان بابور

الموقع الإلكتروني: kau-one.web.app

رقم التواصل: 0530717788

تم دراسه الماده 2026م

تم بحمد الله.